IND320 project work

Part 3: Data quality and patterns

Jupyter Notebook

Sivan Hadji

This notebook continues the same reservoir and Streamlit project. It adds weather data, outlier checks, and time-series pattern views using NVE and Open-Meteo data.

GitHub repository: IND320-Sivan00-Project1

Streamlit app: ind320-sivan00-project1.streamlit.app

Development branch: project-part-3

Preparing the project environment

I use the local course environment for the notebook. The local requirements include the geospatial packages needed to read the NVE area shapefile.

Code

```
from pathlib import Path
import sys
import pandas as pd
import matplotlib.pyplot as plt

# Adding the project folder lets the notebook import the same functions as the app.
PROJECT_DIR = Path.cwd()
if PROJECT_DIR.name != "assignment1" and (PROJECT_DIR / "assignment1").exists():
    PROJECT_DIR = PROJECT_DIR / "assignment1"
sys.path.insert(0, str(PROJECT_DIR))

from quality_utils import (
    build_electricity_sample,
    build_weather_sample,
    fill_spectrogram,
    load_electricity_areas,
    load_nve_regions,
    load_open_meteo_region,
    lof_temperature_plot,
    spc_precipitation_plot,
    stl_decomposition,
)
from data_utils import load_reservoirs

print(f"Project folder: {PROJECT_DIR}")
```

Reading the NVE area polygons

I use the NVE bidding-zone polygons for NO1 to NO5 and the NVE region polygons for RS, RØ, RV, RM and RN. This lets me keep the electricity areas and the regional weather sample separate. The eastern region sample uses RS and RØ, while the northern sample keeps RM and RN available as separate regions.

Code

```
regions = load_nve_regions(PROJECT_DIR / "data" / "areas" / "nve_regions.zip")
electricity_areas = load_electricity_areas(
    PROJECT_DIR / "data" / "areas" / "electricity_areas.zip"
)

print("NVE weather regions:")
display(regions[["reg_code", "reg_name"]])
print("NVE electricity bidding zones:")
display(electricity_areas[["area_code"]])
```

	reg_code	reg_name
0	RS	Region Sør
1	RØ	Region Øst
2	RV	Region Vest
3	RM	Region Midt-Norge
4	RN	Region Nord

	area_code
0	NO1
1	NO2
2	NO3
3	NO4
4	NO5

Sampling locations in each area

I draw 1,000 random points inside each electricity bidding zone and each NVE region polygon. Sampling is done in an equal-area projection, and fixed seeds make the coordinates repeatable. The two samples stay separate because the price zones and NVE regions have different boundaries.

Code

```
# Keeping the point samples in the project makes the API request locations reproducible.
weather_points = build_weather_sample(regions, n_each=1000, seed=320)
electricity_points = build_electricity_sample(electricity_areas, n_each=1000, seed=640)
weather_point_file = PROJECT_DIR / "data" / "areas" / "weather_sample_points.csv"
electricity_point_file = PROJECT_DIR / "data" / "areas" / "electricity_sample_points.csv"
weather_points.to_csv(weather_point_file, index=False)
electricity_points.to_csv(electricity_point_file, index=False)
```

```
print("Weather-region sample sizes:")
display(weather_points.groupby("region_group").size().rename("locations"))
print("Electricity-area sample sizes:")
display(electricity_points.groupby("area_group").size().rename("locations"))
```

```
region_group
RM    1000
RN    1000
RS    1000
RV    1000
RØ    1000
Name: locations, dtype: int64
```

```
area_group
NO1    1000
NO2    1000
NO3    1000
NO4    1000
NO5    1000
Name: locations, dtype: int64
```

Understanding the Open-Meteo request

The Historical Weather API accepts coordinates, a start and end date, hourly variables and a weather model. I use ERA5 and request precipitation and 2 m temperature. The API does not require a token for this non-commercial use. Multiple coordinates are sent in batches so the yearly request stays within a reasonable size.

Code

```
import requests

# This small preview shows the request structure without downloading a full year.
request = requests.Request(
    "GET",
    "https://archive-api.open-meteo.com/v1/archive",
    params={
        "latitude": "63.4,63.5",
        "longitude": "10.4,10.5",
        "start_date": "2024-01-01",
        "end_date": "2024-01-02",
        "hourly": "precipitation,temperature_2m",
        "models": "era5",
        "timezone": "Europe/Oslo",
    },
).prepare()
print(request.url)
```

Downloading NO3 weather for 2024

I use five randomly sampled coordinates from the NO3 bidding-zone polygon for this example. The sample file has 1,000 points for each electricity area, while the weather-region points are stored separately. A full year for every point creates a very large API response, so I average five locations for the 2024 example. The same function can be used with another electricity area and date interval.

Code

```
# Five locations keep the full-year hourly response small enough to rerun reliably.
weather_no3_2024 = load_open_meteo_region(
    "NO3",
    "2024-01-01",
    "2024-12-31",
    point_count=5,
    sample_file=str(electricity_point_file),
)
weather_no3_2024.head()
```

```
time  precipitation  temperature_2m
0 2024-01-01 00:00:00           0.02           -3.06
1 2024-01-01 01:00:00           0.02           -3.10
2 2024-01-01 02:00:00           0.00           -3.30
3 2024-01-01 03:00:00           0.00           -3.28
4 2024-01-01 04:00:00           0.00           -3.38
```

Code

```
# I check the time coverage and missing values before using the hourly averages.
weather_no3_2024.agg({
    "time": ["min", "max", "count"],
    "precipitation": ["mean", "median", "count"],
    "temperature_2m": ["mean", "median", "count"],
})
```

```
time  precipitation  temperature_2m
min     2024-01-01 00:00:00            NaN            NaN
max     2024-12-31 23:00:00            NaN            NaN
count                  8784    8784.000000    8784.000000
mean                    NaN       0.253841       5.173889
median                  NaN       0.100000       4.960000
```

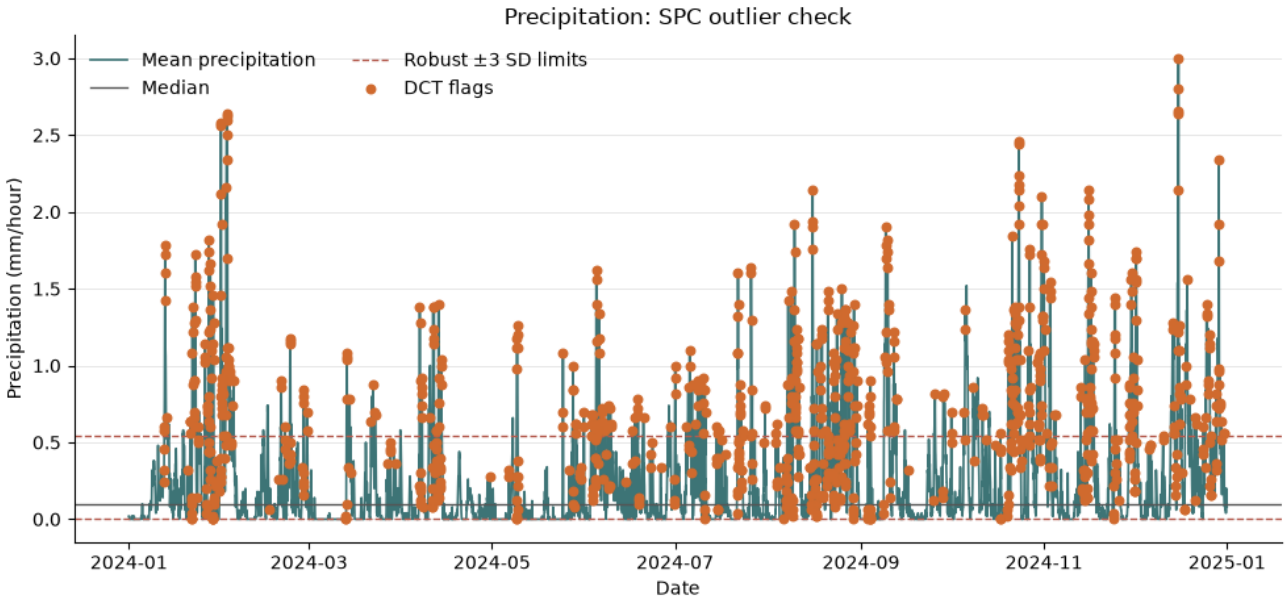
Checking precipitation outliers with SPC

I use a DCT high-pass filter to form SATV, then estimate robust control limits from the full-year data. SATV is only used for detection, so the plot stays on the precipitation scale and makes flagged hours easy to compare with the observed values.

Code

```
# These settings are visible because changing the cutoff or limit changes which hours are flagged.
spc_figure, spc_summary = spc_precipitation_plot(
    weather_no3_2024,
    cutoff=0.08,
    n_sd=3.0,
)
spc_summary
```

```
{'observations': 8784,
 'outliers': 959,
 'outlier_percent': 10.917577413479053,
 'median_mm': 0.1,
 'lower_limit_mm': 0.0,
 'upper_limit_mm': 0.54478,
 'dct_cutoff_fraction': 0.08,
 'robust_sd_multiplier': 3.0}
```



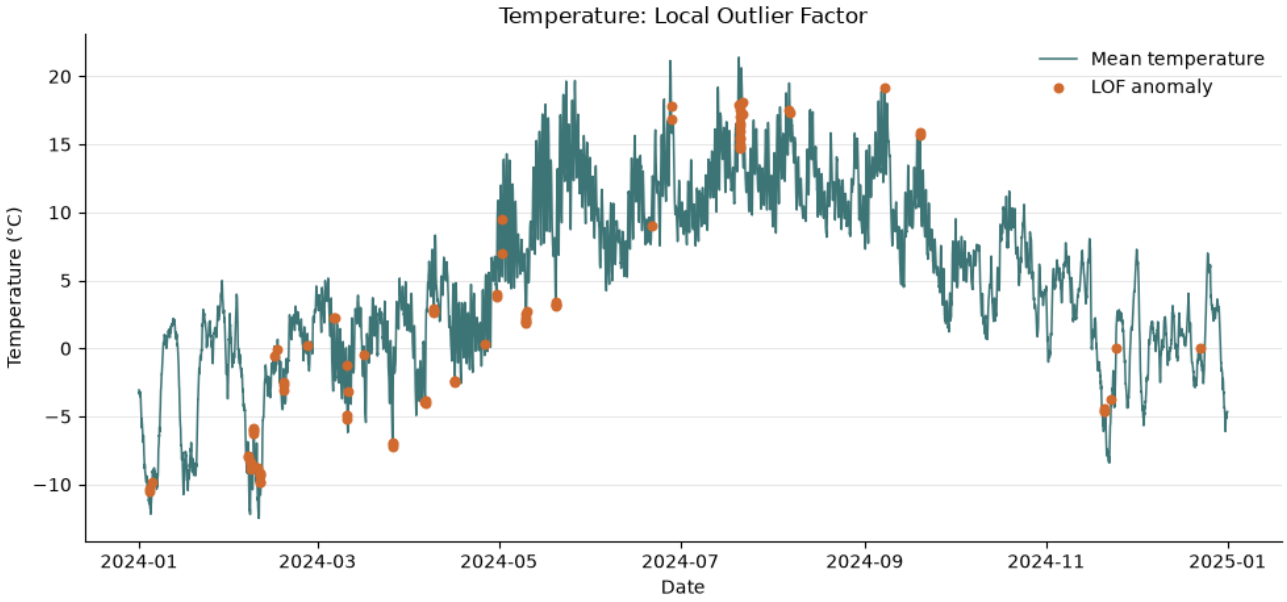
Finding temperature anomalies with LOF

The temperature series has a seasonal cycle, so the LOF function also includes each hour's difference from a rolling 24-hour median. I use a default anomaly proportion of 1%, which can be changed when I compare results.

Code

```
lof_figure, lof_summary = lof_temperature_plot(
    weather_no3_2024,
    contamination=0.01,
)
lof_summary
```

```
{'observations': 8784,
 'anomalies': 88,
 'anomaly_percent': 1.0018214936247722,
 'contamination': 0.01,
 'median_temperature_c': 4.96}
```



Reading the NVE reservoir series

The reservoir data is weekly, while the weather data is hourly. I keep the series at its original time resolution rather than joining hourly values to weekly reservoir observations. For the pattern analysis I select the NVE electricity area NO3.

Code

```
reservoirs = load_reservoirs()
reservoir_no3 = reservoirs.loc[
    (reservoirs["area_type"] == "EL") & (reservoirs["area_number"] == 3)
].sort_values("date")
fill_no3 = reservoir_no3.set_index("date")["fill_fraction"].mul(100)

print(f"NO3 contains {len(fill_no3):,} weekly observations.")
reservoir_no3[["date", "fill_fraction", "capacity_twh", "stored_energy_twh"]].tail()
```

```
date  fill_fraction  capacity_twh  stored_energy_twh
14888 2026-09-20       0.680705      8.920932           6.072518
14897 2026-09-27       0.715288      8.920932           6.381039
14906 2026-10-04       0.718754      8.920932           6.411958
14915 2026-10-11       0.725240      8.920932           6.469815
14924 2026-10-18       0.724940      8.920932           6.467144
```

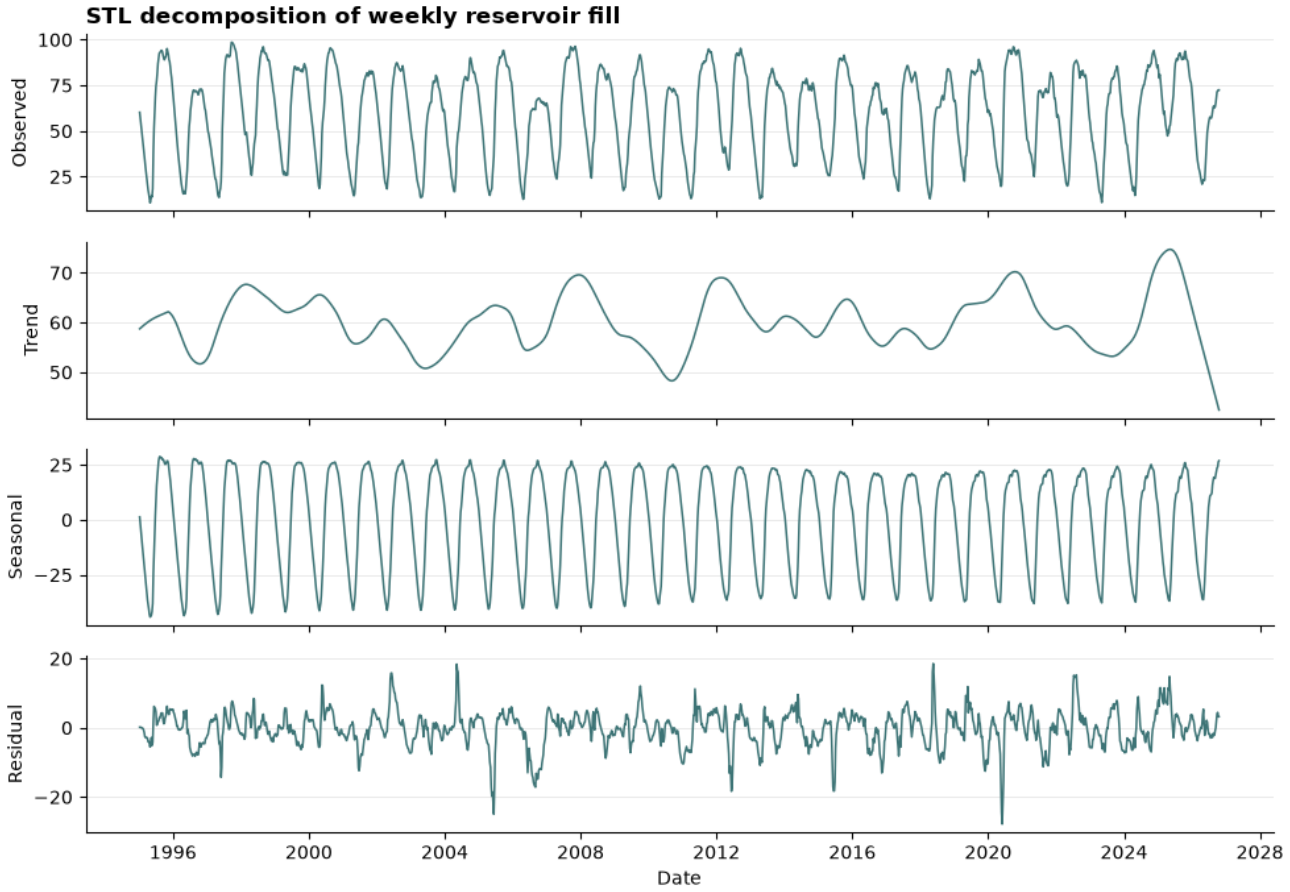
Separating the seasonal pattern with STL

A period of 52 represents a year in a weekly series. STL returns the observed values, trend, seasonal component and residuals. I keep robust fitting on by default because a few unusual weeks should have less influence on the smooth components.

Code

```
stl_figure, stl_components = stl_decomposition(
    fill_no3,
    period=52,
    seasonal_smoother=13,
    trend_smoother=None,
    robust=True,
)
stl_components.tail()
```

```
observed      trend   seasonal  residual
date
2026-09-20  68.070450  44.378710  21.747609  1.944130
2026-09-27  71.528834  43.887901  23.635445  4.005488
2026-10-04  71.875423  43.397389  24.056983  4.421051
2026-10-11  72.523980  42.907204  26.302740  3.314035
2026-10-18  72.494050  42.417366  26.863617  3.213066
```



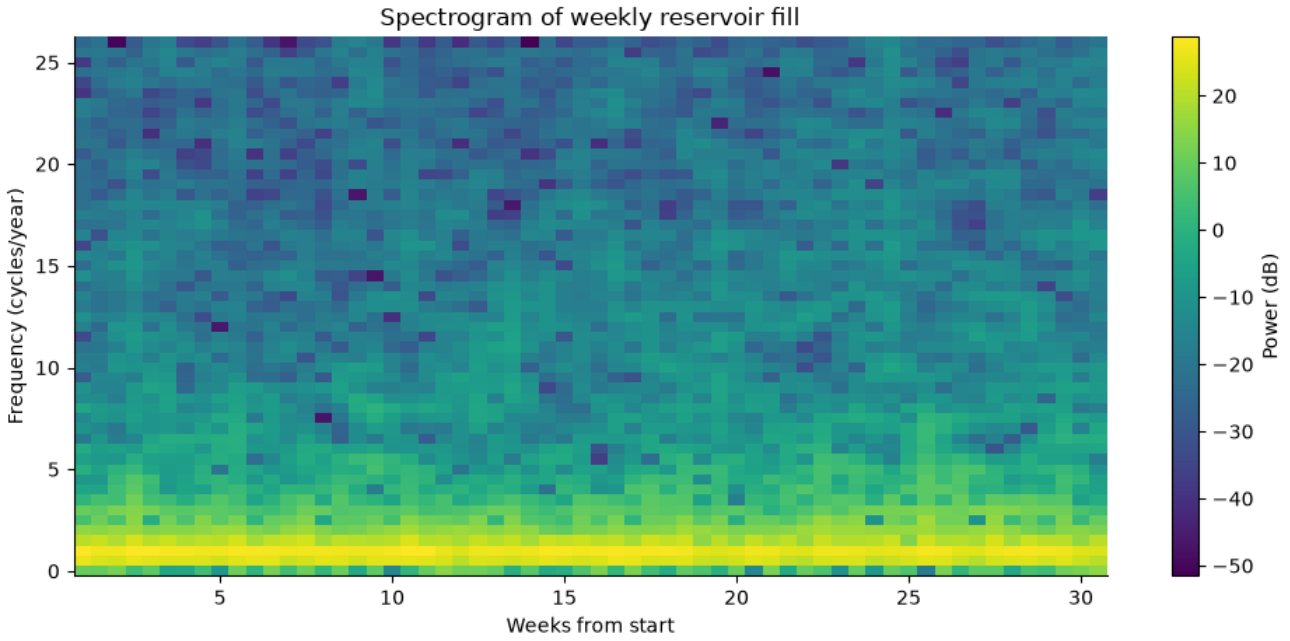
Viewing frequency changes with a spectrogram

The spectrogram uses the same weekly series. Window length controls how much data is used in each frequency estimate, and overlap controls how often a new estimate is made. More overlap gives a smoother view over time, but it does not add new observations.

Code

```
spectrum_figure, spectrum_summary = fill_spectrogram(
    fill_no3,
    window_weeks=104,
    overlap_weeks=78,
)
spectrum_summary
```

```
{'observations': 1659,
 'window_weeks': 104,
 'overlap_weeks': 78,
 'time_windows': 60}
```



Adding the analyses to Streamlit

The app has a reservoir-pattern page with tabs for STL and the spectrogram, and a weather-outlier page with tabs for SPC and LOF. The electricity-area selector uses coordinates sampled inside the matching NO1-NO5 polygon. I keep the separate NVE region sample for describing the regional boundaries, including the RS+RØ east and RM+RN north groups.

AI use

I used AI as learning support to understand API parameters and analysis methods, organise the notebook and app structure, and troubleshoot code. I reviewed the functions and ran the analyses locally so I could check how the parameter choices affect the results.

Work log

In this part I extended the reservoir app with weather data and a few methods for checking patterns and unusual observations. I started by downloading the NVE electricity bidding-zone polygons and NVE region polygons. I made a fixed random sample of 1,000 coordinates in each electricity area and each NVE region. The fixed seeds make the samples repeatable, and keeping the two sets separate matters because the boundaries describe different things.

I used Open-Meteo's Historical Weather API to retrieve hourly ERA5 precipitation and temperature. For the NO3 example, I selected five locations from inside the NO3 bidding-zone polygon for 2024 and averaged their hourly values. I first tried a much larger request, which was slow and hit the API's hourly request limit. I changed the code to request data month by month and use a smaller location sample. The app then loaded 8,784 hourly observations for 2024, including for NO3. The weather data still come from Open-Meteo; the smaller sample is a practical choice for making the request manageable.

For precipitation, I use a DCT high-pass filter to create the SATV signal and use it only to decide which hours to flag. The plot stays on the original precipitation scale, with robust limits estimated from the median and MAD. I also use Local Outlier Factor on temperature, using both the temperature and its difference from a 24-hour rolling median. I left the cutoff and anomaly settings visible because changing them affects which observations are marked.

I use STL to separate the weekly reservoir series into trend, seasonal and residual components. The spectrogram gives another view of repeating patterns, with window length and overlap available as settings. I kept these methods in separate functions so I can reuse them with another series and see how the settings change the result.

I added two pages to the existing Streamlit app. Reservoir patterns has tabs for STL and the spectrogram, while Weather outliers has tabs for precipitation checks and temperature anomalies. I also added an area selector so the weather sample is drawn from the matching electricity-zone polygon. The page reports how many hourly observations were loaded, and the app splits API requests by month so the selected date range can be handled in smaller pieces. I checked the new plots with the NVE reservoir series and with hourly weather data. The changes are on my temporary project-part-3 branch, so they stay together with the earlier parts while I continue working.

Project links

GitHub repository: https://github.com/Sivan00/IND320-Sivan00-Project1

Streamlit app: https://ind320-sivan00-project1.streamlit.app/

The Project 3 work is on the temporary `project-part-3` branch until the peer review and teacher feedback for Part 2 are complete.